

Can you think of any other useful functions to write? Experiment with writing a few parallel computations of your own to see which ones can be expressed without additional primitives. Here are some ideas to try:

- Is there a more general version of the parallel summation function we wrote at the beginning of this chapter? Try using it to find the maximum value of an `IndexedSeq` in parallel.
- Write a function that takes a list of paragraphs (a `List[String]`) and returns the total number of words across all paragraphs, in parallel. Generalize this function as much as possible.
- Implement `map3`, `map4`, and `map5`, in terms of `map2`.

## 7.4 The algebra of an API

As the previous section demonstrates, we often get far just by writing down the type signature for an operation we want, and then “following the types” to an implementation. When working this way, we can almost forget the concrete domain (for instance, when we implemented `map` in terms of `map2` and `unit`) and just focus on lining up types. This isn’t cheating; it’s a natural style of reasoning, analogous to the reasoning one does when simplifying an algebraic equation. We’re treating the API as an *algebra*,<sup>6</sup> or an abstract set of operations along with a set of *laws* or properties we assume to be true, and simply doing formal symbol manipulation following the rules of the game specified by this algebra.

Up until now, we’ve been reasoning somewhat informally about our API. There’s nothing wrong with this, but it can be helpful to take a step back and formalize what laws you expect to hold (or would like to hold) for your API.<sup>7</sup> Without realizing it, you’ve probably mentally built up a model of what properties or laws you expect. Actually writing these down and making them precise can highlight design choices that wouldn’t be otherwise apparent when reasoning informally.

### 7.4.1 The law of mapping

Like any design choice, choosing laws has *consequences*—it places constraints on what the operations can mean, determines what implementation choices are possible, and affects what other properties can be true. Let’s look at an example. We’ll just make up a possible law that seems reasonable. This might be used as a test case if we were writing tests for our library:

```
map(unit(1))(_ + 1) == unit(2)
```

<sup>6</sup> We do mean algebra in the mathematical sense of one or more sets, together with a collection of functions operating on objects of these sets, and a set of *axioms*. Axioms are statements assumed true from which we can derive other *theorems* that must also be true. In our case, the sets are particular types like `Par[A]` and `List[Par[A]]`, and the functions are operations like `map2`, `unit`, and `sequence`.

<sup>7</sup> We’ll have much more to say about this throughout the rest of this book. In the next chapter, we’ll design a declarative testing library that lets us define properties we expect functions to satisfy, and automatically generates test cases to check these properties. And in part 3 we’ll introduce abstract interfaces specified *only* by sets of laws.

We’re saying that mapping over `unit(1)` with the `_ + 1` function is in some sense equivalent to `unit(2)`. (Laws often start out this way, as concrete examples of *identities*<sup>8</sup> we expect to hold.) In what sense are they equivalent? This is an interesting question. For now, let’s say two `Par` objects are equivalent if *for any valid* `ExecutorService` argument, their `Future` results have the same value.

We can check that this holds for a particular `ExecutorService` with a function like this:

```
def equal[A](e: ExecutorService)(p: Par[A], p2: Par[A]): Boolean =
  p(e).get == p2(e).get
```

Laws and functions share much in common. Just as we can generalize functions, we can generalize laws. For instance, the preceding could be generalized this way:

```
map(unit(x))(f) == unit(f(x))
```

Here we’re saying this should hold for *any* choice of `x` and `f`, not just `1` and the `_ + 1` function. This places some constraints on our implementation. Our implementation of `unit` can’t, say, inspect the value it receives and decide to return a parallel computation with a result of `42` when the input is `1`—it can only pass along whatever it receives. Similarly for our `ExecutorService`—when we submit `Callable` objects to it for execution, it can’t make any assumptions or change behavior based on the values it receives. More concretely, this law disallows downcasting or `InstanceOf` checks (often grouped under the term *typecasing*) in the implementations of `map` and `unit`.

Much like we strive to define functions in terms of simpler functions, each of which *do* just one thing, we can define laws in terms of simpler laws that each *say* just one thing. Let’s see if we can simplify this law further. We said we wanted this law to hold for *any* choice of `x` and `f`. Something interesting happens if we substitute the identity function for `f`.<sup>9</sup> We can simplify both sides of the equation and get a new law that’s considerably simpler:<sup>10</sup>

```
map(unit(x))(f) == unit(f(x))  ← Initial law.
map(unit(x))(id) == unit(id(x)) ← Substitute identity function for f.
map(unit(x))(id) == unit(x)    ← Simplify.
map(y)(id) == y                ← Substitute y for unit(x) on both sides.
```

Fascinating! Our new, simpler law talks only about `map`—apparently the mention of `unit` was an extraneous detail. To get some insight into what this new law is saying, let’s think about what `map` *can’t* do. It can’t, say, throw an exception and crash the computation before applying the function to the result (can you see why this violates the law?). All it can do is apply the function `f` to the result of `y`, which of course leaves `y`

<sup>8</sup> Here we mean *identity* in the mathematical sense of a statement that two expressions are identical or equivalent.

<sup>9</sup> The identity function is defined as `def id[A](a: A): A = a`.

<sup>10</sup> This is the same sort of substitution and simplification one might do when solving an algebraic equation.